

Capítulo

RECURSIÓN



4.1 INTRODUCCIÓN

La **recursión** o **recursividad** es un concepto amplio, con muchas variantes, y difícil de precisar con pocas palabras. Aparece en numerosas actividades de la vida diaria; por ejemplo, en una fotografía donde se observa otra fotografía. Otro caso ilustrativo es el que se presenta en los programas de televisión, en los cuales un periodista transfiere el control de la noticia a otro periodista que se encuentra en otra ciudad, y éste, a su vez, hace lo mismo con un tercero. Cuando este último termina su participación regresa el control al segundo, y cuando éste también finaliza su intervención regresa el control al primero. En este capítulo nos limitaremos a tratar la recursión como herramienta de programación.

La recursión es un recurso muy poderoso que permite expresar soluciones simples y naturales a ciertos tipos de problemas. Es importante considerar que no todos los problemas son naturalmente recursivos; algunos sí lo son y otros no.

Un objeto recursivo es aquel que aparece en la definición de sí mismo, así como el que se llama a sí mismo. Los árboles, por ejemplo, que se estudiarán en el capítulo 6, representan las estructuras de datos, no lineales y dinámicas, más eficientes que existen actualmente en computación. La característica de los árboles es que son estructuras inherentemente recursivas. Es decir, en cualquier actividad de programación que se realice con árboles se utiliza la recursividad.

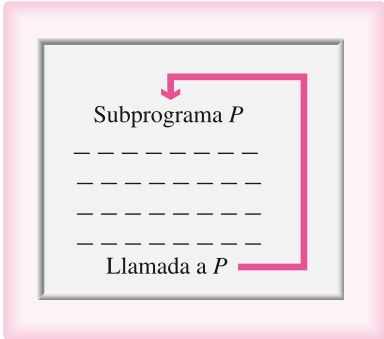
La recursión se puede presentar de dos maneras diferentes:

- a) **Directa:** el programa o subprograma se llama directamente a sí mismo. Por ejemplo, observe que en la figura 4.1 *P* representa un programa y en alguna parte de él aparece una llamada a sí mismo.
- b) **Indirecta:** el subprograma llama a otro subprograma, y éste, en algún momento, llama nuevamente al primero. Por ejemplo, en la figura 4.2 el subprograma *P* llama al subprograma *Q* y éste, a su vez, invoca al primero; es decir, el control regresa a *P*.

En toda definición recursiva de un problema siempre se deben establecer dos pasos diferentes y muy importantes; el **paso básico** y el **paso recursivo**. El primero, uno o varios, dependiendo del problema, se utiliza como condición de parada o fin de la recursividad. A éste llegamos cuando encontramos la solución del problema o cuando decidimos que ya no vamos a seguir, porque no están dadas las condiciones para hacerlo. El

FIGURA 4.1

Recursión directa.



paso segundo, por otra parte, propicia la recursividad. Se pueden presentar uno o varios, nuevamente dependiendo del problema a resolver.

Cuando se analiza la solución recursiva de un problema es importante determinar con precisión cuáles serán los pasos básico y recursivo. En cada vuelta del ciclo es importante que nos acerquemos cada vez más a la solución del problema, o sea, al paso básico. Si esto no ocurre, entonces podemos estar ante un ciclo extraño. Es decir, el problema estaría mal definido y, en tal caso, la máquina se quedaría ejecutando por tiempo indefinido el programa en que estuviera, y sólo terminaría al agotarse la memoria.

A continuación se presentan algunos ejemplos que nos pueden ayudar a comprender más rápidamente el concepto recursión.

Ejemplo 4.1

Factorial de un número

El factorial de un número entero positivo n se define como el producto de los números comprendidos entre 1 y n . También como n por el factorial de $(n - 1)$, así aparece el concepto de recursión. La expresión $n!$ simboliza el factorial de n . A continuación se ilustra este caso.

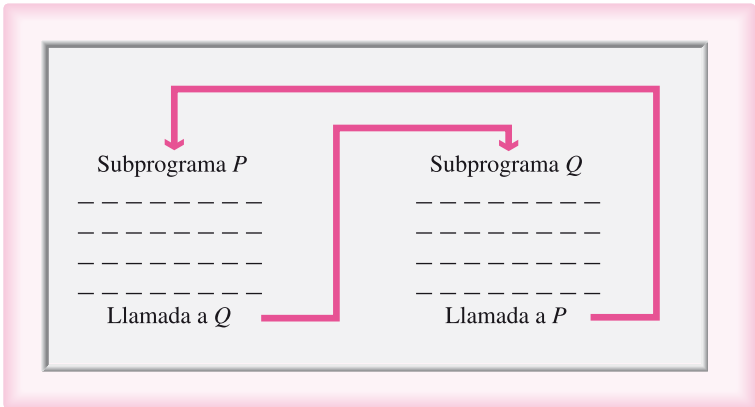
Por definición:

$0! = 1$ → Paso básico

$1! = 1$ → Paso básico

FIGURA 4.2

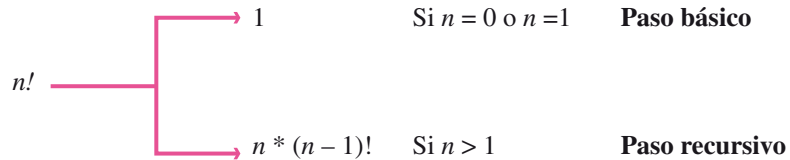
Recursión indirecta.



$$\begin{aligned}
 2! &= 2 * 1! = 2 * 1 \\
 3! &= 3 * 2! = 3 * 2 * 1 \\
 4! &= 4 * 3! = 4 * 3 * 2 * 1 \\
 &\vdots \\
 n! &= n * (n-1)! = n * (n-1) * \dots * 4 * 3 * 2 * 1 \quad \rightarrow \text{Paso recursivo}
 \end{aligned}$$

Al calcular, por ejemplo, el factorial de 4, decimos que $4!$ es igual a $4 * 3!$. Al factorial de 3, lo calculamos como $3 * 2!$. Al factorial de 2, como $2 * 1!$, y así sucesivamente hasta llegar a un paso básico que detiene la recursividad. Llegamos a definir, entonces, el factorial de un número n en términos del factorial del número $(n-1)$.

Fórmula 4.1



A continuación se presenta el algoritmo recursivo del cálculo del factorial.

Algoritmo 4.1 Factorial_rec

Factorial_rec (N)

{Este algoritmo calcula el factorial de un número N en forma recursiva, donde N es un valor numérico entero, positivo o nulo}

1. Si ($N = 0$)
 entonces
 Hacer Factorial_rec $\leftarrow 1$ {Paso básico}
 si no
 Hacer Factorial_rec $\leftarrow N * \text{Factorial_rec}(N-1)$
 {Llamada —paso— recursiva}
2. {Fin del condicional del paso 1}

Cuando se realiza una llamada recursiva, se utiliza, en forma implícita, una estructura tipo pila para almacenar las instrucciones pendientes de ejecutar, con todos los valores que tienen las variables o constantes en ese momento. Cuando se termina la ejecución se llega al estado básico, se toma la instrucción que se encuentra en el tope de la pila y se continúa operando. Esta acción se repite hasta que la pila quede vacía.

La siguiente tabla presenta los valores que van adquiriendo las variables, durante el cálculo de $4!$.

TABLA 4.1

Cálculo del factorial
en forma iterativa

N	FACT
4	1
3	4
2	12
1	24
0	24

Ejemplo 4.2

Sucesión de Fibonacci. Otro problema clásico de recursividad es el del cálculo de la sucesión de Leonardo de Pisa, conocido como Fibonacci:

0, 1, 1, 2, 3, 5, 8, 13, 21,..., etcétera.

Algunas propiedades de esta sucesión, según Wikipedia, la enciclopedia libre, son:

- ▶ El cociente entre un término y el inmediatamente anterior varía continuamente, pero se estabiliza en un número irracional conocido como *razón áurea* o número áureo, que es la solución positiva de la ecuación $x^2 - x - 1 = 0$, y se puede aproximar por 1618033989.
- ▶ Cualquier número natural se puede escribir mediante la suma de un número limitado de términos de la sucesión de Fibonacci, cada uno de ellos distinto a los demás. Por ejemplo, $17 = 13 + 3 + 1$, $65 = 55 + 8 + 2$.
- ▶ Tan sólo un término de cada tres es par, uno de cada cuatro es múltiplo de 3, uno de cada cinco es múltiplo de 5, etc. Esto se puede generalizar, de forma que la sucesión de Fibonacci es periódica en las congruencias módulo m , para cualquier m .
- ▶ Si Fibonacci de p , $F(p)$, es un número primo, p también lo es, con una única excepción: $F(4) = 3$; 3 es primo, pero 4 no lo es.
- ▶ La suma infinita de los términos de la sucesión $F(n)/10^n$ es exactamente $10/89$.

El Fibonacci de un número se obtiene de la suma de los dos números Fibonacci anteriores. Por definición:

$$\begin{aligned}
 \text{Fibonacci}(0) &= 0 && \rightarrow \text{Paso básico} \\
 \text{Fibonacci}(1) &= 1 && \rightarrow \text{Paso básico} \\
 \text{Fibonacci}(2) &= \text{Fibonacci}(1) + \text{Fibonacci}(0) \\
 &= 1 + 0 = 1 \\
 \text{Fibonacci}(3) &= \text{Fibonacci}(2) + \text{Fibonacci}(1) \\
 &= 1 + 1 = 2 \\
 \text{Fibonacci}(4) &= \text{Fibonacci}(3) + \text{Fibonacci}(2) \\
 &= 2 + 1 = 3 \\
 &\dots \\
 \text{Fibonacci}(n) &= \text{Fibonacci}(n-1) + \text{Fibonacci}(n-2) && \rightarrow \text{Paso recursivo}
 \end{aligned}$$

La fórmula 4.2 presenta una definición recursiva de la serie de Fibonacci.

Fórmula 4.2

$$\text{Fibonacci}(n) = \begin{cases} n & \text{si } (n = 0) \text{ o } (n = 1) \\ \text{Fibonacci}(n-1) + \text{Fibonacci}(n-2) & \text{si } n > 1 \end{cases}$$

En este ejemplo el paso básico se presenta cuando $n = 1$ o $n = 0$. En el paso recursivo de la fórmula se utiliza el concepto de Fibonacci aplicado a $(n-1)$ y $(n-2)$. Por ser n un número entero positivo serán $(n-1)$ y $(n-2)$ valores más cercanos al estado básico.

El algoritmo 4.3 presenta una solución recursiva del cálculo de un número Fibonacci n .

Algoritmo 4.3 Fibonacci_rec

Fibonacci_rec(N)

{Este algoritmo calcula el número Fibonacci correspondiente a N en forma recursiva, donde N es un valor numérico entero, positivo o nulo}

1. Si $((N = 0) \text{ o } (N = 1))$

entonces

Hacer Fibonacci_rec $\leftarrow N$ {Paso básico}

si no

Hacer Fibonacci_rec $\leftarrow$ Fibonacci_rec($N-1$) + Fibonacci_rec($N-2$)
{Llamadas recursivas}

2. {Fin del condicional del paso 1}

En la figura 4.4 se puede observar el seguimiento del algoritmo para $N = 6$.

Observe que en la pila el número que se encuentra entre corchetes hace referencia a la llamada recursiva que se realiza. Por ejemplo, [1] expresa que ésa es la primera llamada recursiva, [2] la segunda llamada y así sucesivamente. Por otra parte, el mismo número entre corchetes a la izquierda, Fibonacci_rec(N), relaciona la llamada recursiva con el valor que ingresa al algoritmo en cada llamada.

Fibonacci representa el caso de un algoritmo que se puede resolver naturalmente de manera recursiva, pero que resulta ineficiente hacerlo de esta forma tanto en cuanto a tiempo como a espacio que se utiliza en la pila para almacenar las llamadas pendientes de ejecutar. Además, observe que en muchas ocasiones se tiene que resolver el mismo

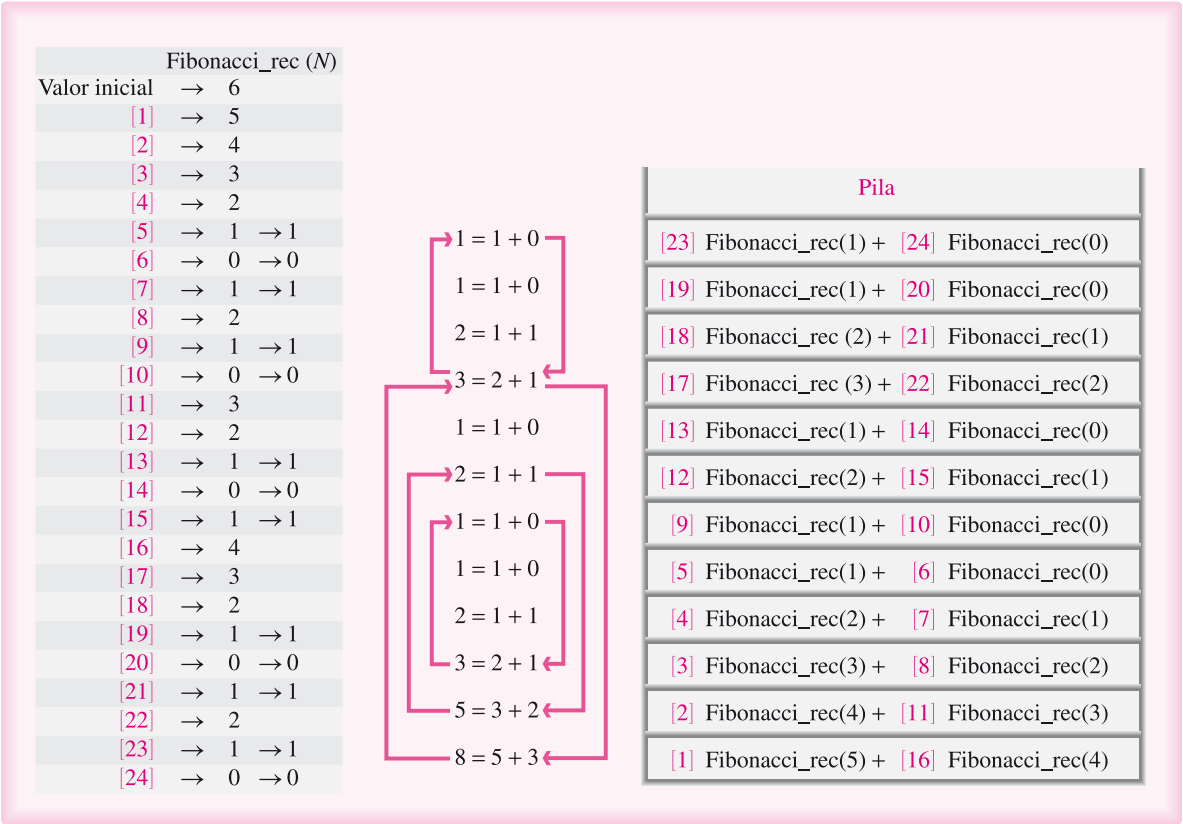


FIGURA 4.4
Funcionamiento interno de la recursión: números Fibonacci.

problema, aun cuando ya tengamos una solución para ese caso, y esto resulta completamente impráctico además de ineficiente. En el ejemplo que se muestra en la figura 4.4, Fibonacci_rec(2) se tuvo que resolver en cinco ocasiones, llamadas recursivas 4, 8, 12, 18 y 22.

A continuación se presenta una variante iterativa para resolver el problema de los números de Fibonacci.

Algoritmo 4.4 Fibonacci_ite

Fibonacci_ite (N)

{Este algoritmo calcula el número Fibonacci correspondiente a N en forma iterativa. N es un valor numérico entero, positivo o nulo}

{FIBO, FIBA, FIBB e I son variables de tipo entero}

1. Si $((N = 0) \text{ o } (N = 1))$
entonces
 Hacer $FIBO \leftarrow N$
si no
 Hacer $FIBA \leftarrow 0, FIBB \leftarrow 1 \text{ e } I \leftarrow 2$
2. {Fin del condicional del paso 1}
3. Mientras $(I \leq N)$ Repetir
 Hacer $FIBO \leftarrow FIBB + FIBA, FIBA \leftarrow FIBB, FIBB \leftarrow FIBO \text{ e } I \leftarrow I + 1$
4. {Fin del ciclo del paso 3}
5. Escribir FIBO

En la tabla 4.2 se presentan los valores que van adquiriendo las variables durante el cálculo del número Fibonacci $N = 6$.

TABLA 4.2
Cálculo de los números
Fibonacci en forma iterativa

<i>N</i>	<i>FIBO</i>	<i>FIBA</i>	<i>FIBB</i>	<i>I</i>
6		0	1	2
	1	1	1	3
	2	1	2	4
	3	2	3	5
	5	3	5	6
	8	5	8	7

Ejemplo 4.3

Impresión de un arreglo

Dado como dato un arreglo unidimensional de tipo entero, escriba el contenido de las casillas del mismo de **izquierda a derecha**.

A continuación se presenta el algoritmo correspondiente.

Algoritmo 4.5 Arreglo_imp

Arreglo_imp (*A*, *N*)

{El algoritmo escribe de izquierda a derecha los elementos de un arreglo unidimensional *A* de tipo entero. *N* representa el tamaño del arreglo}

1. Si $(N \neq 0)$ entonces
 Arreglo_imp (*A*, $N - 1$)
 Escribir *A*[*N*]
2. {Fin del condicional del paso 1}

FIGURA 4.5

Funcionamiento interno de la recursión: impresión de arreglos.

A	8	12	21	27	31	44	48
---	---	----	----	----	----	----	----

Arreglo_imp (A, N)	Impresión de resultados	
A 7	8	E [1]
A 6	12	E [2]
A 5	21	E [3]
A 4	27	E [4]
A 3	31	E [5]
A 2	44	E [6]
A 1	48	E [7]
A 0		

Pila	
Escribir A [1]	E [1]
Escribir A [2]	E [2]
Escribir A [3]	E [3]
Escribir A [4]	E [4]
Escribir A [5]	E [5]
Escribir A [6]	E [6]
Escribir A [7]	E [7]

En la figura 4.5 se puede observar el seguimiento del algoritmo para un arreglo unidimensional de tipo entero de siete elementos.

Observe que $E[N]$ en la pila se utiliza para mostrar el orden en que se realizan las llamadas recursivas. Por otra parte, en *Impresión de resultados*, el mismo símbolo se utiliza para relacionar la llamada recursiva con el valor que se imprime.

Ejemplo 4.4

Impresión de un arreglo

Dado como dato un arreglo unidimensional de tipo entero, escriba el contenido de las casillas del mismo de **derecha a izquierda**.

A continuación se presenta el algoritmo correspondiente.

Algoritmo 4.6 Arreglo_imp

Arreglo_imp (A, N)

{El algoritmo escribe de derecha a izquierda los elementos de un arreglo unidimensional A de tipo entero. N representa el tamaño del arreglo}

1. Si ($N \neq 0$) entonces
Escribir A[N]
Arreglo_imp (A, N - 1)
2. {Fin del condicional del paso 1}

Es importante señalar que, en este caso, a diferencia del algoritmo 4.5, no se utiliza la pila —internamente— para ir guardando instrucciones pendientes de ejecución. Luego de evaluarse la condición, si ésta resulta verdadera se escribe el valor de la posición N

del arreglo y se llama nuevamente a la función, ahora con $N - 1$. Una vez que se alcanza el estado básico, el algoritmo termina.

Ejemplo 4.5

Suma de un arreglo

Dado como dato un arreglo unidimensional de tipo entero, obtenga la suma del mismo. A continuación se presenta el algoritmo correspondiente.

Algoritmo 4.7 Arreglo_sum

Arreglo_sum (A, N)

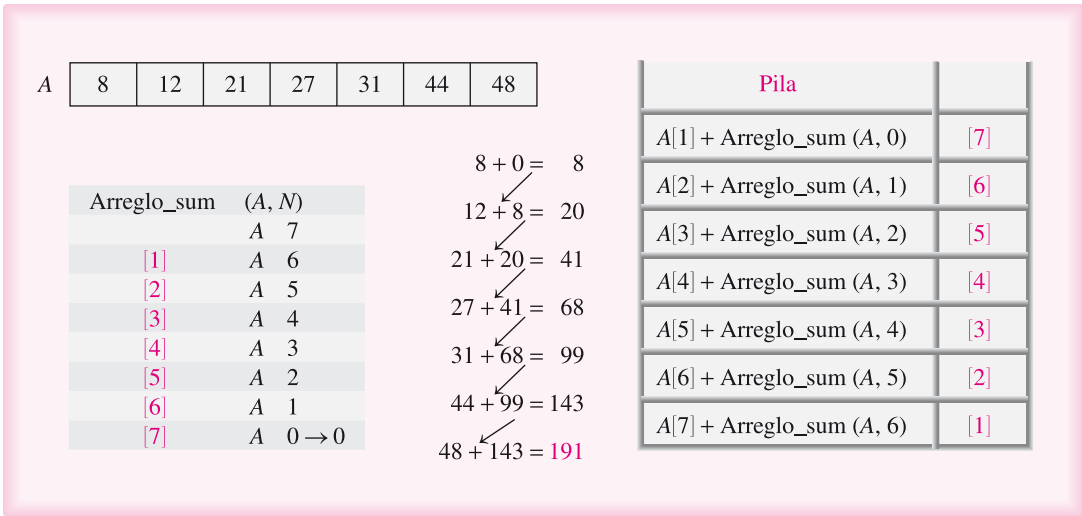
{El algoritmo obtiene la suma de los elementos de un arreglo unidimensional A de tipo entero. N representa el tamaño del arreglo}

- 1. Si (N = 0)
 entonces
 Hacer Arreglo_sum $\leftarrow$ 0
 si no
 Hacer Arreglo_sum $\leftarrow$ A[N] + Arreglo_sum(A, N - 1)
- 2. {Fin del condicional del paso 1}

En la figura 4.6 se observa el seguimiento del algoritmo para un arreglo unidimensional de tipo entero de siete elementos.

FIGURA 4.6

Funcionamiento interno de la recursión: suma de arreglos.



Observe nuevamente que el número entre corchetes en la *Pila*, se utiliza para mostrar el orden con que se realizan las llamadas recursivas. El número entre corchetes que se encuentra en *Arreglo_sum*(A, N) relaciona la llamada recursiva con los valores que recibe la función.

Ejemplo 4.6

Euclides

El algoritmo de Euclides representa un método efectivo para encontrar el máximo común divisor (mcd) entre dos números enteros positivos. El algoritmo consiste en varias divisiones euclidianas sucesivas. En la primera división se toma como dividendo el mayor de los números y como divisor el otro ahorrando así un paso. Luego el divisor y el resto sirven, respectivamente, de dividendo y divisor de la siguiente división. El proceso se detiene cuando se obtiene un resto nulo.

La fórmula 4.3 presenta la definición recursiva del algoritmo de Euclides. Es importante mencionar que debe cumplirse que $M \geq N$. A continuación se presenta el algoritmo correspondiente:

Fórmula 4.3

Euclides (M, N)

M Si $N = 0$
 $\text{Euclides}(N, M \bmod N)$ En cualquier otro caso

Algoritmo 4.8 Euclides

Euclides (M, N)

{Este algoritmo calcula el máximo común divisor de dos números enteros positivos. M y N son valores numéricos enteros positivos}

1. Si ($N = 0$)
 entonces
 Hacer Euclides $\leftarrow M$
 si no
 Hacer Euclides $\leftarrow \text{Euclides}(N, M \bmod N)$
2. {Fin del condicional del paso 1}

En la figura 4.7 se observa el seguimiento del algoritmo para dos corridas diferentes. En la primera, $M = 2\,353$ y $N = 1\,651$. En la segunda, $M = 25\,680$ y $N = 11\,892$.

Es importante señalar que, en este caso, no se utiliza la pila internamente para ir guardando instrucciones pendientes de ejecución. Una vez que se alcanza el estado básico, el algoritmo termina dando el resultado final.

FIGURA 4.7
Funcionamiento interno de la recursión: algoritmo de Euclides.

Euclides	(M, N)
2 353	1 651
1 651	702 [1]
702	247 [2]
247	208 [3]
208	39 [4]
39	13 [5]
13	0 [6] → 13

Euclides	(M, N)
25 680	11 892
11 892	1 896 [1]
1 896	516 [2]
516	348 [3]
348	168 [4]
168	12 [5]
12	0 [6] → 12

Ejemplo 4.7

Ackermann

La función de Ackermann, utilizada en la teoría de la computación, es una función recursiva que toma dos números naturales como argumentos y devuelve un número natural. La fórmula 4.4 presenta la definición recursiva de la función de Ackermann:

$$A(m,n)=\begin{cases} n+1, & \text{si } m=0; \\ A(m-1,1), & \text{si } m>0 \text{ y } n=0; \\ A(m-1,A(m,n-1)), & \text{si } m>0 \text{ y } n>0 \end{cases}$$

Fórmula 4.4

Según Wikipedia, la función crece rápidamente. Para darse una idea de la magnitud de los valores que aparecen de la fila 4 en adelante cuando *m* es igual a 4, se puede destacar que, por ejemplo, *A*(4, 2) es mayor que el número de partículas que forman el universo elevado a la potencia 200 y el resultado de *A*(5, 2) no se puede escribir dado que no cabría en el universo físico. En general, por encima de la fila 4 ya no es posible escribir todos los dígitos del resultado de la función.

En la tabla 4.3 se presenta, con el objeto de que el lector pueda observar la complejidad de la función cuando los valores se incrementan, el resultado de la función de Ackermann y el nivel de profundidad que se alcanza cuando *M* y *N* toman ciertos valores.

TABLA 4.3
Ackermann y valores de *M* y *N*

M	N	Ackermann	Profundidad
1	0	2	2
2	0	3	5
3	0	5	15
2	1	5	14

(continúa)

TABLA 4.3
(continuación)

M	N	Ackermann	Profundidad
2	2	7	27
2	3	9	44
2	4	11	65
2	7	17	152
3	5	253	42 438
3	6	509	172 233
3	10	8 189	44 698 325

A continuación se presenta el algoritmo que resuelve el problema de Ackermann.

Algoritmo 4.9 Ackermann

Ackermann (M, N)

{Este algoritmo calcula la función de Ackermann. M y N son valores numéricos enteros, positivos o nulos}

1. Si ($M = 0$)
 entonces
 Hacer Ackermann $\leftarrow (N + 1)$ {Estado básico}
 si no
 1.1 Si ($N = 0$)
 entonces
 Hacer Ackermann $\leftarrow$ Ackermann ($M - 1, 1$)
 si no
 Hacer Ackermann $\leftarrow$ Ackermann ($M - 1$, Ackermann ($M, N - 1$))
 1.2 {Fin del condicional del paso 1.1}
 2. {Fin del condicional del paso 1}

En la figura 4.8 se puede observar el seguimiento del algoritmo de Ackermann para valores de $M = 2$ y $N = 2$.

Considere nuevamente que el número entre corchetes, tanto en la pila como en $Ackermann(N)$, se utiliza para observar el orden con que se realizan las llamadas recursivas.

Ejemplo 4.8

Algoritmo de partición

Este algoritmo, recientemente descubierto, permite conocer todas las formas en las cuales un número entero positivo puede ser descompuesto como la suma de varios suman-

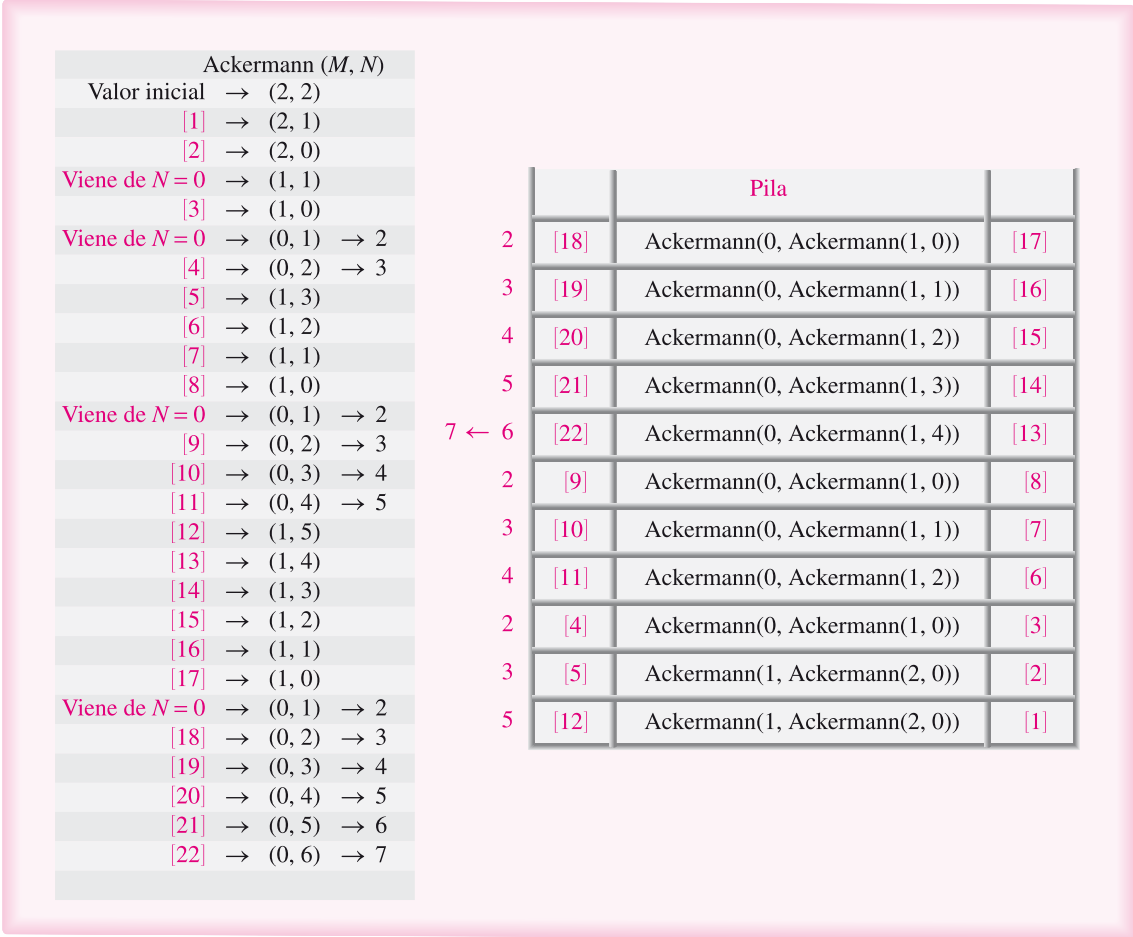


FIGURA 4.8
Funcionamiento interno de la recursión: función de Ackermann.

dos. La solución encontrada a este problema puede servir tanto a la física de partículas como a la seguridad informática.

Según el *New Scientist*, Karl Mahlburg, estudiante de la Universidad de Wisconsin, ha dedicado un año para solucionar el problema que implica trabajar con patrones de números. “He llenado de cálculos y ecuaciones cuaderno tras cuaderno”, dice Mahlburg.

Los patrones fueron descubiertos por primera vez por Ramanujan (1887-1920), un hindú que fue expulsado de la universidad por descuidar los estudios de todo lo que no fueran matemáticas. Su pasión por las matemáticas le llevó a seguir investigando y a escribir a matemáticos ingleses exponiendo sus teorías. Autodidacto, empezó a trabajar en el *Madras Port Trust*, y luego, en 1914, fue admitido en la Universidad de Cambridge. Posteriormente fue elegido miembro de la *Royal Society* y del *Trinity College*. Pocos años después regresó a la India, donde falleció tras una misteriosa enfermedad.

A continuación se presenta la fórmula correspondiente.

Fórmula 4.5



A continuación se presenta el algoritmo correspondiente.

Algoritmo 4.10 Partición

Partición (M, N)

{Este algoritmo calcula de cuantas formas diferentes se puede descomponer un número entero positivo. M y N son valores numéricos enteros positivos. Al iniciar el algoritmo $M = N$ }

1. Si $((M = 1) \text{ o } (N = 1))$
 entonces
 Hacer Partición $\leftarrow 1$ {Estado básico}
 si no
 - 1.1 Si $(M < N)$
 entonces
 Hacer Partición (M, M)
 si no
 - 1.1.1 Si $(M = N)$
 entonces
 Hacer Partición $\leftarrow 1 + \text{Partición}(M, M - 1)$
 si no
 Hacer Partición $\leftarrow \text{Partición}(M, N - 1) + \text{Partición}(M - N, N)$
 - 1.1.2 {Fin del condicional del paso 1.1.1}
 - 1.2 {Fin del condicional del paso 1.1}
2. {Fin del condicional del paso 1}

En la figura 4.9 se presenta el seguimiento del algoritmo para obtener el número de formas en que se puede descomponer el número 6, $M = N = 6$.

Observe nuevamente que el número entre corchetes tanto en la pila como en $\text{Partición}(M, N)$ se utiliza para observar el orden con que se realizan las llamadas recursivas.

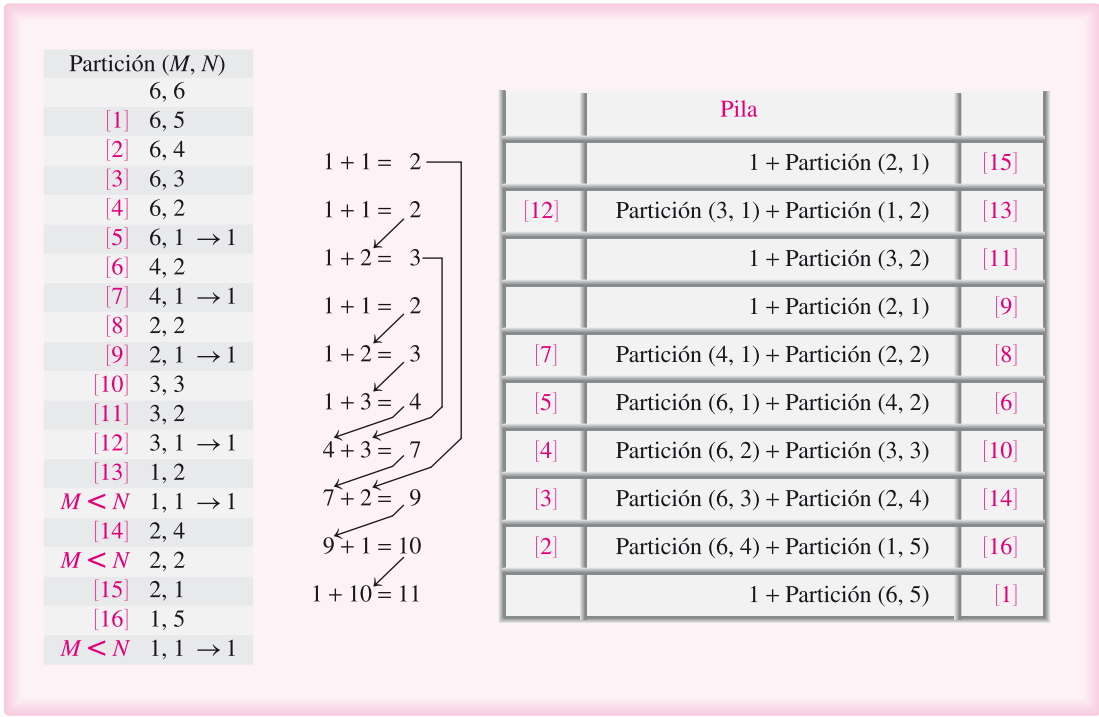


FIGURA 4.9
Funcionamiento interno de la recursión: algoritmo de partición.

Ejemplo 4.9 Los números de Catalan

Estos números se utilizan en una gran variedad de problemas de combinatoria. Tienen varias aplicaciones; por ejemplo, dados como datos n matrices, permiten encontrar el número de formas en que se podrían multiplicar. Otra aplicación consiste en determinar el número de formas en que un polígono con $n + 2$ lados se puede descomponer en n triángulos.

En combinatoria los números de Catalan forman una secuencia de números naturales que aparece en varios problemas de conteo que habitualmente son recursivos. Obtienen su nombre del matemático belga Eugène Charles Catalan (1814-1894).

El enésimo número de Catalan se obtiene según la siguiente fórmula:

$$C_n = \frac{1}{n+1} \binom{2n}{n} \quad \text{con } n \geq 0$$

Fórmula 4.6

Una manera recursiva de expresar los números de Catalan se observa en la siguiente fórmula.

Fórmula 4.7

$$\text{Catalan}(N) = \begin{cases} 1 & \text{Si } N = 1 \\ \sum_{I=1}^{N-1} \text{Catalan}(I) * \text{Catalan}(N-I) & \text{En cualquier otro caso} \end{cases}$$

Los primeros números de Catalan siguiendo la fórmula recursiva son:

1, 1, 2, 5, 14, 42, 132, 429, 1 430,...

El problema, sin embargo, lo representa la gran cantidad de llamadas recursivas que se necesitan realizar para alcanzar estos resultados. Por ejemplo, si $N = 5$ que arrojaría como valor el número 14, se necesitaría casi una centena de llamadas recursivas.

A continuación se presenta el algoritmo que resuelve este problema.

Algoritmo 4.11 Catalan

Catalan (N)

{Este algoritmo obtiene el resultado de los números de Catalan. N es un valor numérico entero positivo.}

{ I y S son variables de tipo entero}

1. Si ($N = 1$)
 entonces
 Hacer Catalan $\leftarrow 1$ {Estado básico}
 si no
 Hacer $S \leftarrow 0$
 - 1.1 Repetir con I desde 1 hasta N
 Hacer Catalan $\leftarrow S + \text{Catalan}(I) * \text{Catalan}(N - I)$
 - 1.2 {Fin del ciclo del paso 1.1}
2. {Fin del condicional del paso 1}

La figura 4.10 muestra el seguimiento del algoritmo de Catalan para $N = 3$.

Ejemplo 4.10

Coeficientes binomiales

El triángulo de Pascal es un triángulo de números enteros, infinito y simétrico cuyas diez primeras líneas se pueden observar en la figura 4.11:

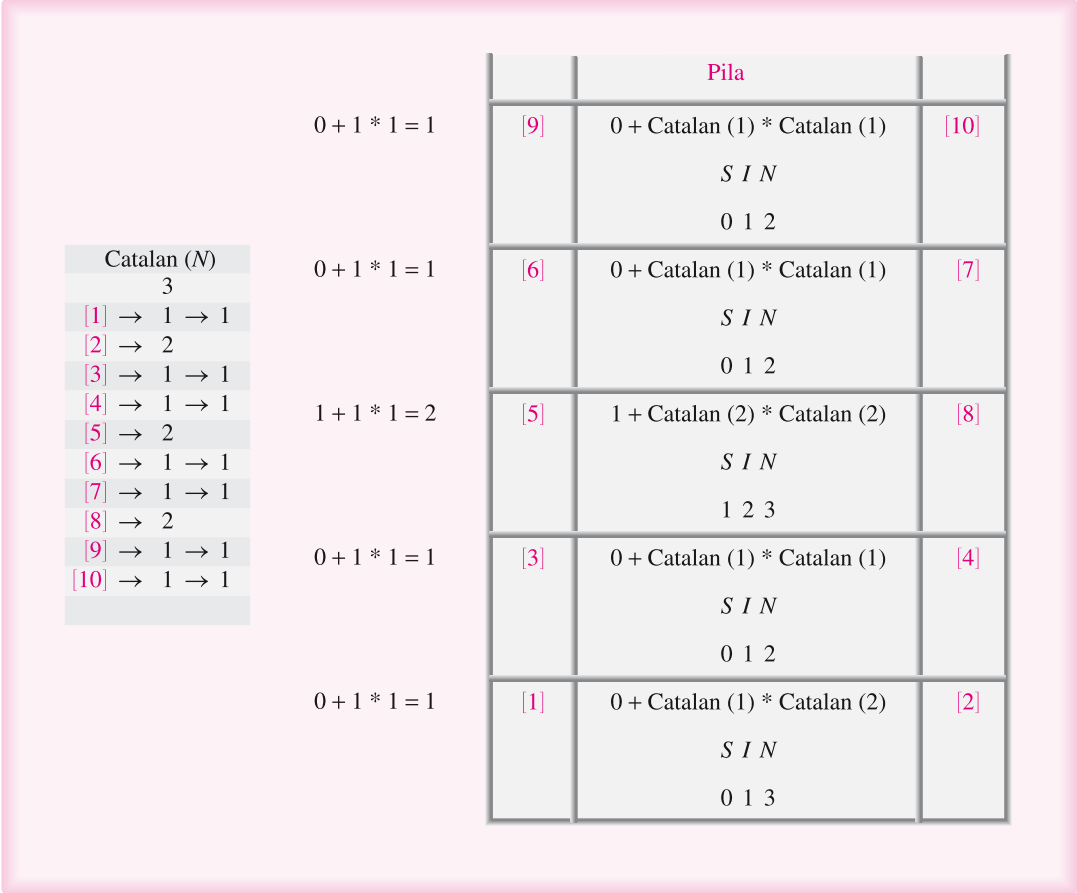


FIGURA 4.10

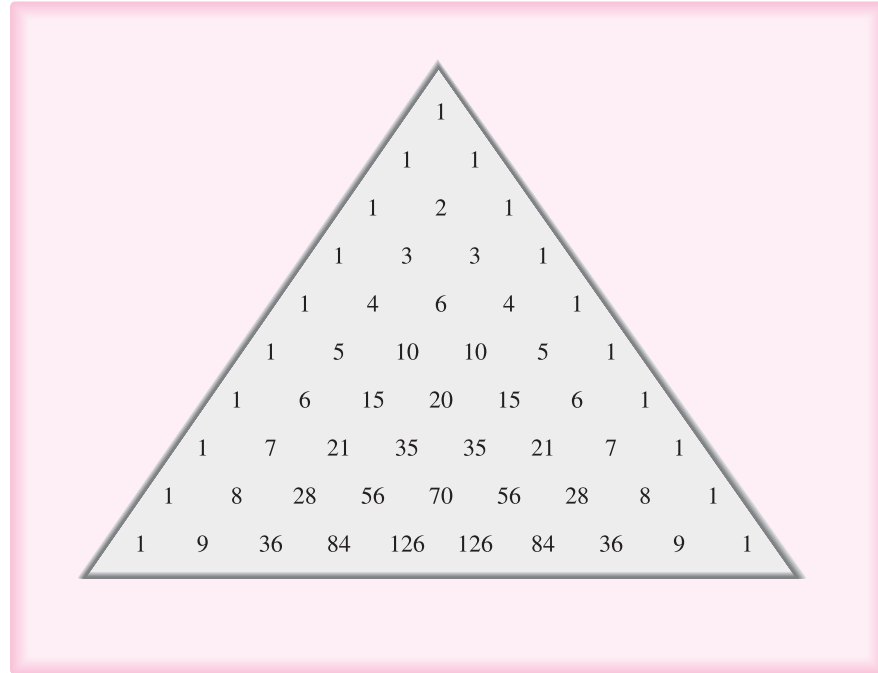
Funcionamiento interno de la recursión: los números de Catalan.

El triángulo se construye de la siguiente manera: primero se escribe el 1 del primer renglón, la cima del triángulo. A partir de la siguiente línea entre cada número se deja un número determinado de espacios en blanco, tres en este caso, para dar claridad. Cada número que se escribe en un renglón es la suma de los dos que se encuentran en el renglón de arriba. Por ejemplo, el 2 que se encuentra en medio del segundo renglón representa la suma de los dos números que se encuentran arriba de él. El 6 del quinto renglón se deriva de la suma de los dos números de arriba, 3 y 3. El primer 10 del sexto renglón se deriva de la suma de los dos números que se encuentran arriba, 4 y 6, y así sucesivamente.

Se puede observar, además, que los lados exteriores del triángulo están formados por 1. Si le quitamos el lado del costado izquierdo, entonces nos quedan los números naturales en orden creciente del 1 al 9. Podemos hacer lo mismo con el otro costado, ya que existe un eje de simetría vertical que pasa por el vértice del triángulo.

FIGURA 4.11

Triángulo de Pascal.



La fórmula que da el desarrollo de $(a + b)^n$ según las potencias crecientes de a y decrecientes de b se llama binomio de Newton. En esta expresión lo único que se desconoce son los coeficientes de los monomios $a^k b^{n-k}$. La definición habitual de los coeficientes binomiales se expresa en términos de factoriales, como se puede observar en la fórmula 4.8.

$$C_n^k = \frac{n!}{k!(n-k)!} (0 \leq k \leq n)$$

Fórmula 4.8

Sin embargo, también es posible presentar una definición recursiva, como se muestra en la siguiente fórmula.

$$CB(N, K) = \begin{cases} 1 & \text{Si } CB(N, 0) \text{ o } C(N, N) \\ CB(N-1, K) + CB(N-1, K-1) & \text{Si } N > K > 0 \end{cases}$$

Fórmula 4.9

El algoritmo 4.12 describe la solución del problema de los coeficientes binomiales.

Algoritmo 4.12 CB

CB (N, K)

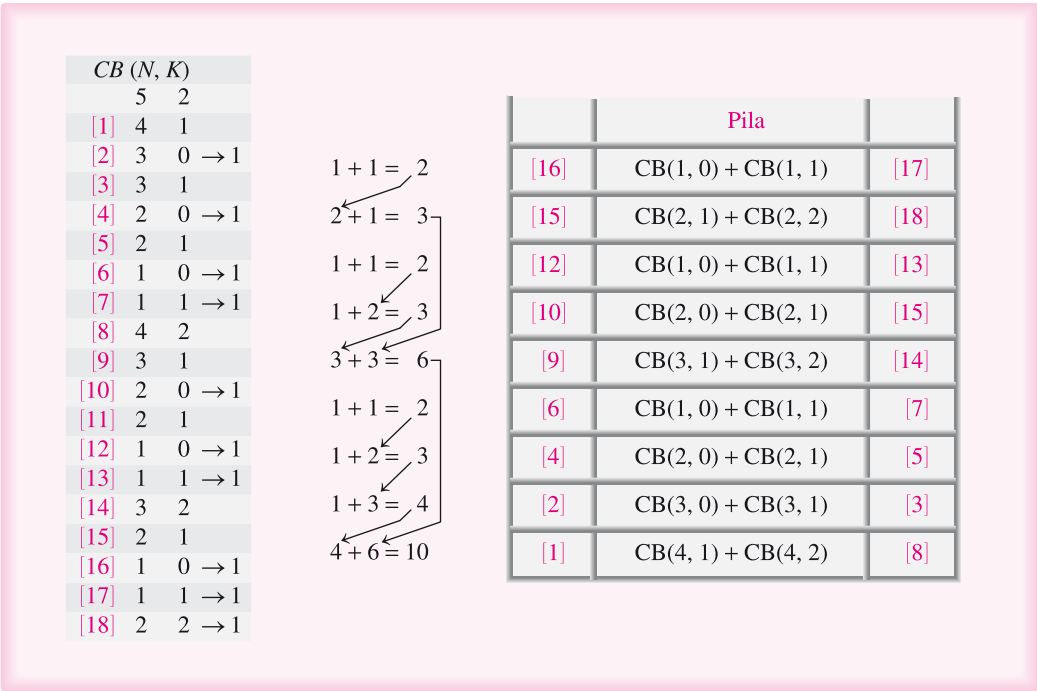
{Este algoritmo obtiene el resultado de los coeficientes binomiales. *N* y *K* son valores numéricos enteros positivos o nulos}

1. Si ((*K* = 0) o (*N* = *K*))
 entonces
 Hacer *CB* ← 1 {Estado básico}
 si no
 Hacer *CB* ← *CB*(*N* − 1, *K* − 1) + *CB*(*N* − 1, *K*)
2. {Fin del condicional del paso 1}

En la figura 4.12, se puede observar el seguimiento del algoritmo para *N* = 5 y *K* = 2.
No olvide que los números que aparecen entre corchetes son para ilustrar el orden en que se realizan las llamadas recursivas.

FIGURA 4.12

Funcionamiento interno de la recursión: coeficientes binomiales.



4.2 EL PROBLEMA DE LAS TORRES DE HANOI

El problema de las Torres de Hanoi es un problema clásico de recursión, ya que pertenece a la clase de problemas cuya solución se simplifica notablemente al utilizar este concepto.

Se tienen tres torres y un conjunto de N discos de diferentes tamaños. Cada uno de ellos tiene una perforación en el centro que le permite deslizarse por cualquiera de las torres. Inicialmente los N discos están ordenados de mayor a menor en una de las torres.

El objetivo del problema consiste en pasar los N discos de la torre de origen a una torre destino, utilizando la otra torre disponible, como auxiliar. A continuación se presentan las reglas que se deben respetar en cada movimiento.

1. En cada movimiento sólo puede intervenir un disco; por lo tanto, siempre será el disco superior el que pueda moverse.
2. No puede quedar un disco sobre otro de menor tamaño.

Supongamos que las torres se identifican con los nombres A , B y C . Los discos inicialmente se encuentran en la torre A —origen—. El objetivo, como señalamos anteriormente, consiste en transferir todos los discos a la torre B —destino—, utilizando la torre C como auxiliar. En las figuras 4.13a y b se presentan el estado inicial y el estado final, respectivamente, del problema de las Torres de Hanoi para cinco discos.

Es importante observar, si se analiza detenidamente el problema, que éste se puede descomponer en tres subproblemas, uno de los cuales, el segundo, se considera trivialidad porque implica únicamente un movimiento. Se muestran a continuación los diferentes subproblemas:

1. Transferir $(N - 1)$ discos de la torre A —origen— a la torre C —auxiliar—.
2. Mover un disco de la torre A —origen— a la torre B —destino—.
3. Transferir $(N - 1)$ discos de la torre C —auxiliar— a la torre B —destino—.

En la figura 4.14 se presenta la solución al problema de las Torres de Hanoi para dos discos, $N = 2$.

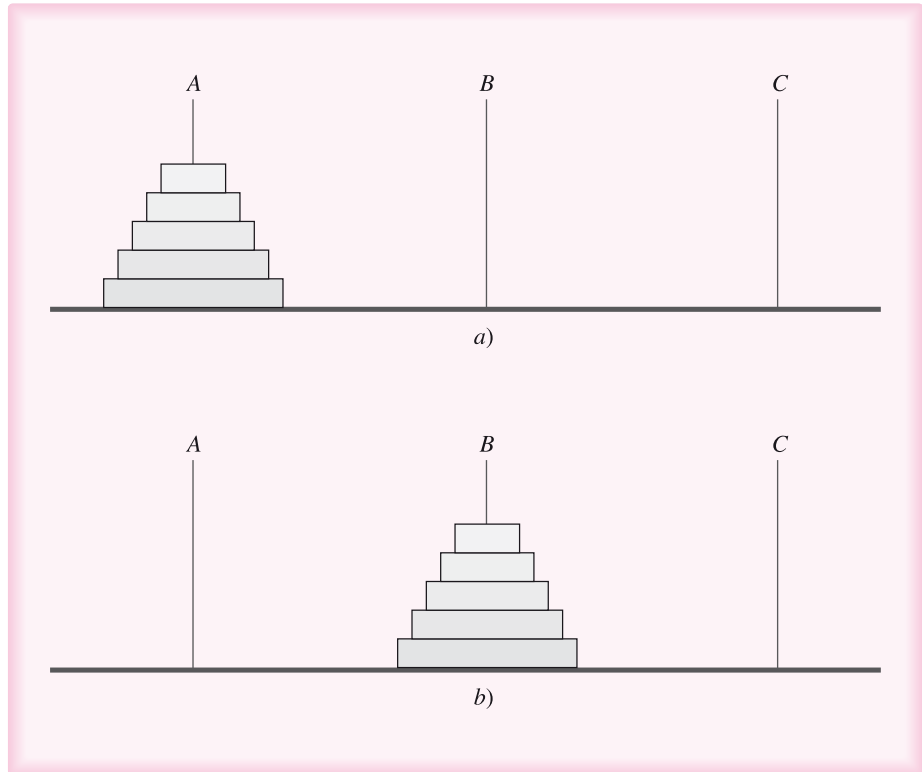
Observe que se realizan tres movimientos:

Mover de A a C
 Mover de A a B
 Mover de C a B

En la figura 4.15, por otra parte, se ilustra la solución del problema de las Torres de Hanoi para tres discos, $N = 3$. Primero se transfieren dos $(N - 1)$ discos de la torre A a la torre auxiliar C (figuras 4.15b, c y d). Posteriormente se realiza el movimiento del disco de la torre A a la torre destino B (figura 4.15e). Finalmente se resuelve el tercer subproblema, se transfieren $(N - 1)$ discos de la torre C —auxiliar— a la torre A (figuras 4.15f, g, h).

FIGURA 4.13

Torres de Hanoi.
a) Estado inicial.
b) Estado final.



Los movimientos que se realizan para resolver este problema son: mover de A a C

Mover de A a B
Mover de B a C
Mover de A a B
Mover de C a A
Mover de C a B
Mover de A a B

Luego de realizar numerosas pruebas para distintos valores de N , se puede concluir que para cualquier N , el número de movimientos (NM) está dado por la siguiente fórmula:

$$NM = 2^n - 1$$

Fórmula 4.10

Así, por ejemplo, para cinco discos se efectuarán 31 movimientos, para diez discos 1 023 movimientos y para 15 discos 32 767 movimientos.

A continuación se presenta el algoritmo recursivo que permite resolver este problema.

FIGURA 4.14

Torres de Hanoi ($N = 2$).
a) Estado inicial. b) Luego de mover un disco de A a C. c) Luego de mover un disco de A a B. d) Estado final.

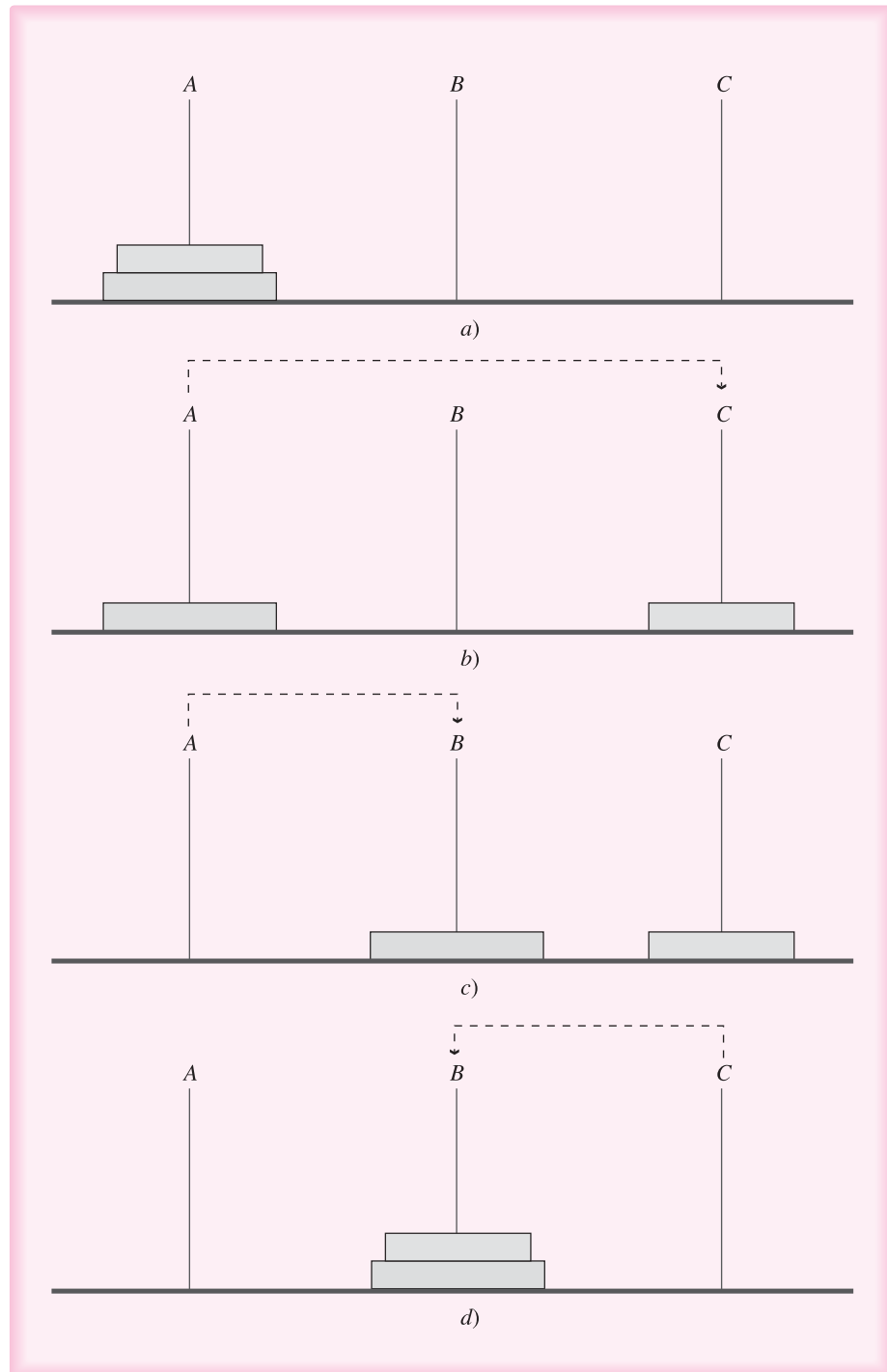


FIGURA 4.15

Torres de Hanoi ($N = 3$).
a) Estado inicial.
b) Luego de mover un disco de A a B.
c) Luego de mover un disco de A a C.
d) Luego de mover un disco de B a C.

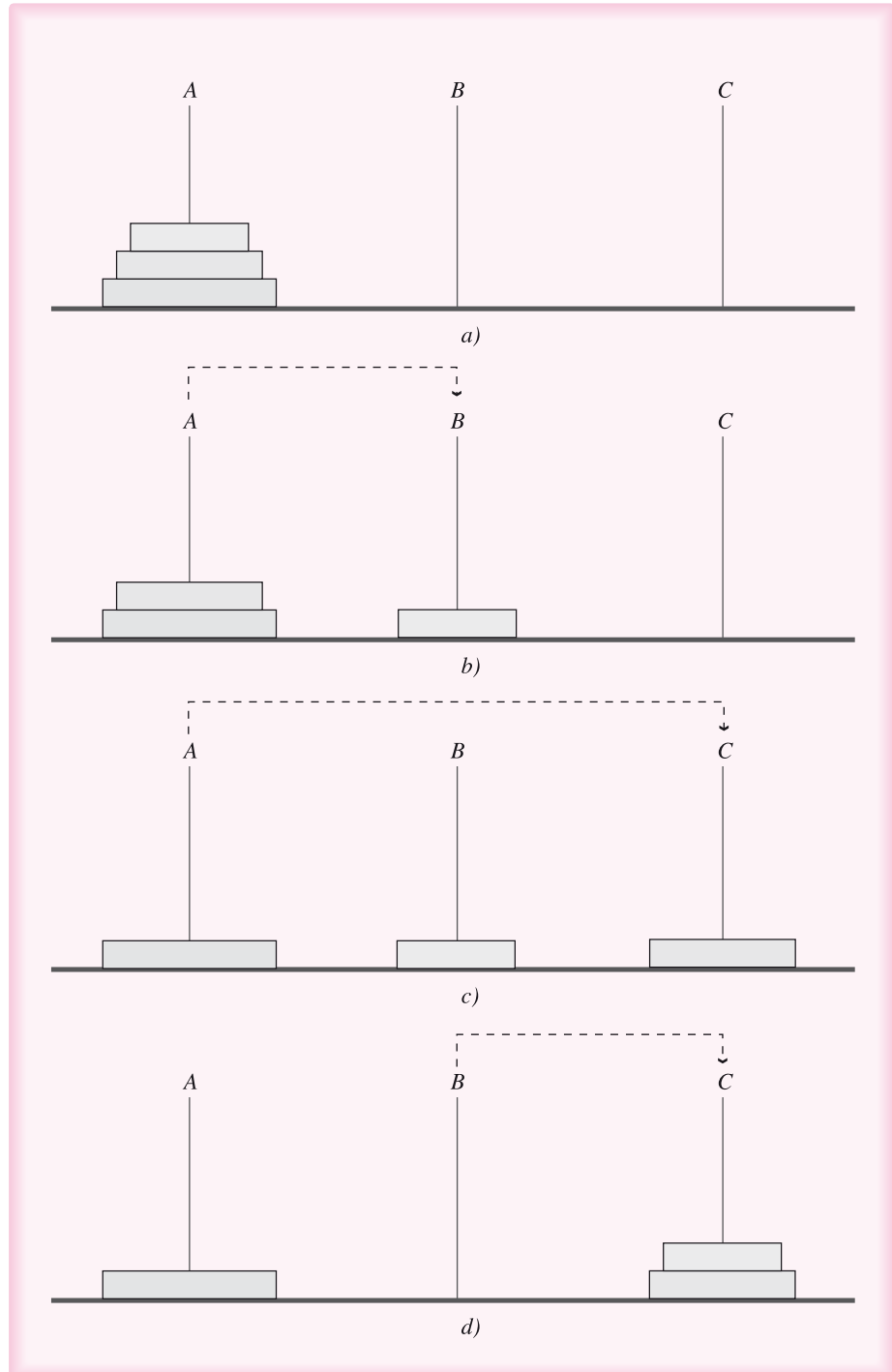
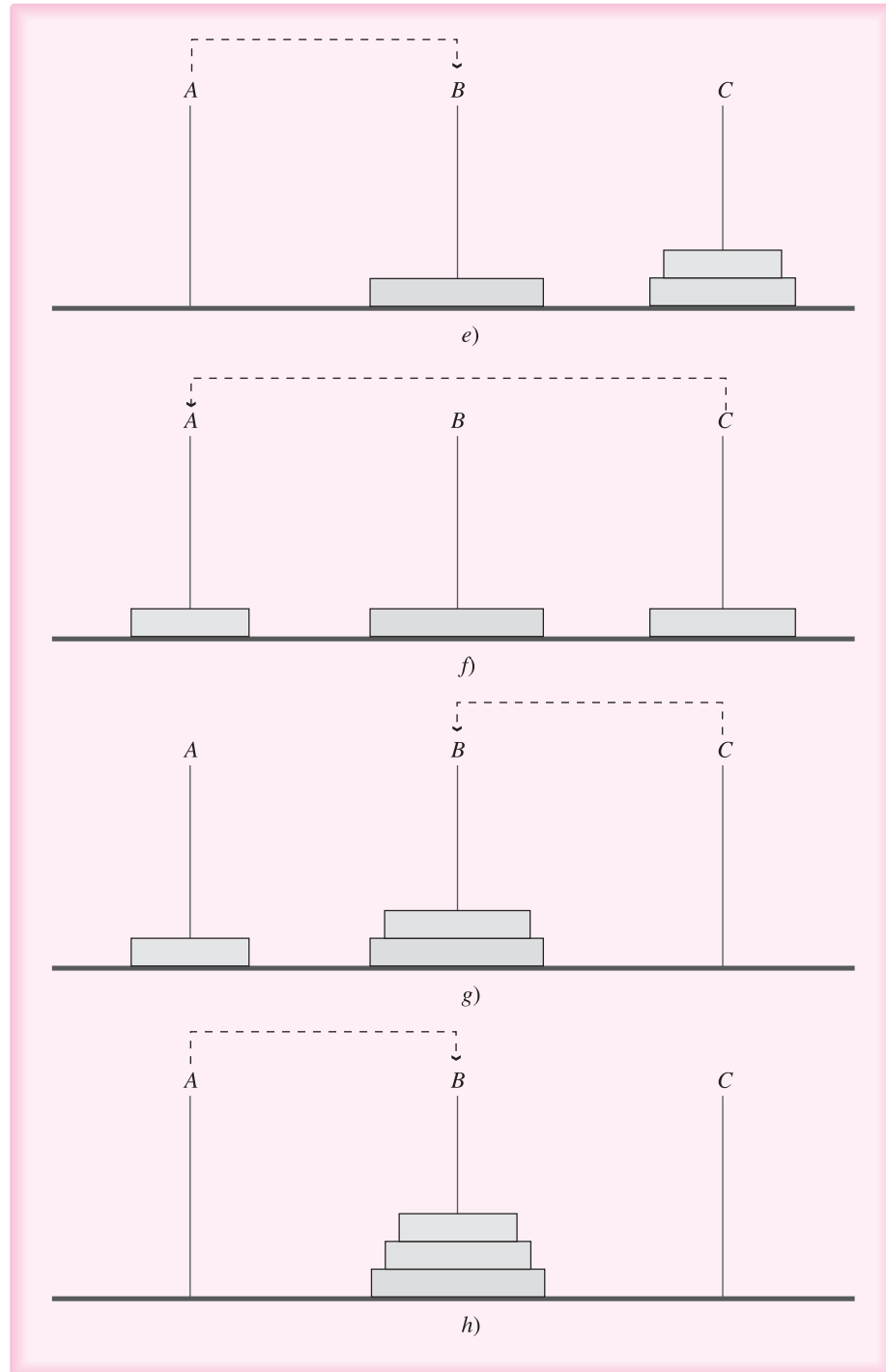


FIGURA 4.15

(continuación)
 e) Luego de mover un disco de A a B.
 f) Luego de mover un disco de C a A.
 g) Luego de mover un disco de C a B.
 h) Estado final.



Algoritmo 4.13 Hanoi

Hanoi (N, ORIGEN, DESTINO, AUXILIAR)

{Este algoritmo obtiene y escribe los movimientos que se deben realizar para transferir los N discos de la torre ORIGEN a la torre DESTINO, con ayuda de la torre AUXILIAR }

1. Si ($N = 1$)
 entonces
 Escribir “Mover un disco de ORIGEN a DESTINO” {Estado básico}
 si no
 {Mover $N - 1$ discos de la torre ORIGEN a la torre AUXILIAR}
 Hanoi ($N - 1$, ORIGEN, AUXILIAR, DESTINO)
 Escribir “Mover un disco de ORIGEN a DESTINO”
 {Mover $N - 1$ discos de la torre AUXILIAR a la torre DESTINO}
 Hanoi ($N - 1$, AUXILIAR, DESTINO, ORIGEN)
2. {Fin del condicional del paso 1}

En la figura 4.16 se muestra el seguimiento del algoritmo para cuatro discos.

FIGURA 4.16

Funcionamiento interno de la recursión: torres de Hanoi.

Hanoi (N, A, B, C)				
4	A	B	C	
3	A	C	B	
2	A	B	C	
1	A	C	B	[M1] : A → C
[1]	1	C	B	A [M3] : C → B
[2]	2	B	C	A
1	B	A	C	[M5] : B → A
[3]	1	A	C	B [M7] : A → C
[4]	3	C	B	A
2	C	A	B	
1	C	B	A	[M9] : C → B
[5]	1	B	A	C [M11] : B → A
[6]	2	A	B	C
1	A	C	B	[M13] : A → C
[7]	1	C	B	A [M15] : C → B

Movimientos			
[M1] A → C	[M9] C → B		
[M2] A → B	[M10] C → A		
[M3] C → B	[M11] B → A		
[M4] A → C	[M12] C → B		
[M5] B → A	[M13] A → C		
[M6] B → C	[M14] A → B		
[M7] A → C	[M15] C → B		
[M8] A → B			

	Pila	
	Escribir A → B	M [14]
[7]	1 C B A	
	Escribir C → A	M [10]
[5]	1 B A C	
	Escribir C → B	M [12]
[6]	2 A B C	
	Escribir B → C	M [6]
[3]	1 A C B	
	Escribir A → B	M [2]
[1]	1 C B A	
	Escribir A → C	M [4]
[2]	2 B C A	
	Escribir A → B	M [8]
[4]	3 C B A	

El algoritmo recursivo anterior ofrece una solución clara y compacta al problema de las Torres de Hanoi. Sin embargo, es posible también presentar una solución iterativa a este problema, pero es conveniente mencionar que en este caso es más complicada y extensa.

Un subprograma generalmente trabaja con variables locales y parámetros. Cuando se hace una llamada recursiva al subprograma, los valores actuales de variables y parámetros se deben conservar. Además se debe almacenar la dirección a la cual se tendrá que regresar el control una vez que se termina de ejecutar.

El algoritmo anterior trabaja con cuatro parámetros: N , ORIGEN, DESTINO y AUXILIAR. Para construir el algoritmo iterativo, si se quieren conservar los valores de los parámetros en cada llamada recursiva, se deberá definir una pila para cada uno de ellos. Otra alternativa sería definir una única pila, en la que cada elemento fuera capaz de almacenar los cuatro parámetros. En el algoritmo que se va a presentar se trabaja con cuatro pilas:

PILAN: para almacenar imágenes de N .
 PILAO: para almacenar imágenes de ORIGEN.
 PILAD: para almacenar imágenes de DESTINO.
 PILAX: para almacenar imágenes de AUXILIAR.

También será necesario manejar un TOPE para las pilas. El siguiente algoritmo presenta una solución iterativa al problema de las Torres de Hanoi.

Algoritmo 4.14 Hanoi_ite

Hanoi_ite (N , ORIGEN, DESTINO, AUXILIAR)

{Este algoritmo resuelve el problema de las Torres de Hanoi de manera no recursiva. ORIGEN, DESTINO y AUXILIAR son parámetros que representan las tres torres. N simboliza el número de discos}

{PILAN, PILAO, PILAD y PILAX son estructuras de datos tipo pila. TOPE es una variable de tipo entero. VARAUX es una variable de tipo carácter. BAND es una variable de tipo booleano}

1. Hacer $TOPE \leftarrow 0$ y $BAND \leftarrow \text{FALSO}$
2. Mientras $((N > 0) \text{ y } (BAND = \text{FALSO}))$ Repetir
 - 2.1 Mientras $(N > 1)$ Repetir
 - Hacer $TOPE \leftarrow TOPE + 1$, $PILAN[TOPE] \leftarrow N$, $PILAO[TOPE] \leftarrow \text{ORIGEN}$,
 $PILAD[TOPE] \leftarrow \text{DESTINO}$, $PILAX[TOPE] \leftarrow \text{AUXILIAR}$
 - {Simula llamada a Hanoi con $N - 1$, ORIGEN, AUXILIAR y DESTINO}
 - Hacer $N \leftarrow N - 1$, $VARAUX \leftarrow \text{DESTINO}$, $DESTINO \leftarrow \text{AUXILIAR}$
 y $AUXILIAR \leftarrow VARAUX$
 - 2.2 {Fin del ciclo del paso 2.1}
 - Escribir "Mover un disco de ORIGEN a DESTINO"
 - Hacer $BAND \leftarrow \text{VERDADERO}$
 - 2.3 Si $(TOPE > 0)$ {Las pilas no están vacías}

entonces

{ Se extraen los elementos del TOPE de la pila }

Hacer $N \leftarrow \text{PILAN}[\text{TOPE}]$, $\text{ORIGEN} \leftarrow \text{PILAO}[\text{TOPE}]$,
 $\text{DESTINO} \leftarrow \text{PILAD}[\text{TOPE}]$, $\text{AUXILIAR} \leftarrow \text{PILAA}[\text{TOPE}]$
 y $\text{TOPE} \leftarrow \text{TOPE} - 1$
 Escribir “Mover un disco de ORIGEN a DESTINO”
{ Simula llamada a Hanoi con $N - 1$, AUXILIAR, DESTINO y ORIGEN }

Hacer $N \leftarrow N - 1$, $\text{VARAUX} \leftarrow \text{ORIGEN}$, $\text{ORIGEN} \leftarrow \text{AUXILIAR}$,
 $\text{AUXILIAR} \leftarrow \text{VARAUX}$ y $\text{BAND} \leftarrow \text{FALSO}$

2.4 { Fin del condicional del paso 2.3 }

3. { Fin del ciclo del paso 2 }

En la figura 4.17 se muestra el seguimiento del algoritmo para tres discos.

FIGURA 4.17.
Seguimiento del algoritmo
iterativo: Torres de Hanoi.

N	ORIGEN	DESTINO	AUXILIAR	TOPE	BAND	VARAUX
3	A	B	C	0	F	
2		C	B	1		B
1		B	C	2	V	C
2	A	C	B	1		
1	B		A		F	A
3	A	B	C	0	V	
2	C		A		F	A
1		A	B	1	V	B
2	C	B	A	0		
1	A		C		F	C
					V	

Nota: en el lugar de VERDADERO se escribe V,
en el lugar de FALSO F.

Movimientos		Pila N	Pila O	Pila D	Pila A
$A \rightarrow B$					
$A \rightarrow C$					
$B \rightarrow C$					
$A \rightarrow B$					
$C \rightarrow A$					
$C \rightarrow B$					
$A \rightarrow B$					
$A \rightarrow C$					
Tope $\rightarrow$ 1		2	C	B	A
Tope $\rightarrow$ 2		2	A	C	B
Tope $\rightarrow$ 1		3	A	B	C

4.3 RECURSIVIDAD EN ÁRBOLES

Los árboles representan las estructuras de datos dinámicas y no lineales más interesantes de computación. La estructura de **árboles balanceados** o **AVL** es la estructura de datos más eficiente para trabajar en la memoria rápida de la computadora. Por otra parte, la estructura de datos **Árboles-B**, representa la estructura de datos más eficiente para trabajar en memoria secundaria —dispositivos de almacenamiento secundario—.

Los árboles son una estructura inherentemente recursiva y todas las operaciones que se realizan en árboles se deben programar en forma recursiva. A diferencia de otras estructuras de datos en las cuales las operaciones se pueden implementar tanto en forma recursiva como iterativa, independientemente de las diferencias que se pueden observar con respecto a la eficiencia, en los árboles sólo se puede trabajar de manera recursiva.

En el capítulo correspondiente a árboles el lector podrá practicar en forma intensa la recursividad.

4.4 RECURSIVIDAD EN ORDENACIÓN Y BÚSQUEDA

En los capítulos 8 y 9 el lector podrá aplicar nuevamente el concepto de recursividad. En el capítulo 8 se presenta el método más eficiente de ordenación, *Quicksort*, que funciona de manera recursiva.

En el capítulo 9, correspondiente a búsqueda, el lector podrá estudiar nuevamente gran cantidad de algoritmos recursivos.

 EJERCICIOS

1. **Inversión de capital.** Se ha depositado en una institución bancaria un monto de capital m por el cual se recibe un $X\%$ de interés anual. El problema consiste en determinar el capital que se tendrá al cabo de n años. Escriba un subprograma recursivo que resuelva este problema. Recuerde que debe establecer los estados básico y recursivo del problema.
2. Retome el problema anterior y resuélvalo de manera iterativa. Compare sus soluciones, teniendo en cuenta la eficiencia en el manejo de memoria y la legibilidad del código generado.
3. Escriba un subprograma recursivo que invierta el orden de los elementos de un arreglo de N números enteros. Es decir, que el elemento que está en la posición 1 se intercambie con el que está en la posición N , el de la posición 2, con el de la $N - 1$, y así sucesivamente.
4. Retome el problema anterior y resuélvalo de manera iterativa. Compare sus soluciones, teniendo en cuenta la eficiencia en el manejo de memoria y la legibilidad del código generado.
5. Escriba un subprograma recursivo que invierta el orden de una cadena de caracteres. Por ejemplo, si la cadena de entrada es ROMA, el resultado que debe arrojar el programa es AMOR.
6. Se tienen tres arreglos: SUR, CENTRO y NORTE que almacenan los nombres de los países de Sur, Centro y Norteamérica, respectivamente. Los tres arreglos están ordenados en forma alfabética. Escriba un subprograma recursivo que mezcle los tres arreglos anteriores, formando un cuarto arreglo, AMÉRICA, en el cual aparezcan los nombres de todos los países del continente ordenados alfabéticamente. Compare esta solución con la desarrollada para el problema 8 del capítulo 1.
7. Dado como dato el siguiente programa, sígalo y diga qué imprime para los siguientes valores de X : $X = 38$, $X = 51$, $X = 24$

P1 (X)

{ X es un parámetro de tipo entero positivo}

1. Si ($X > 100$)

entonces

$P1 \leftarrow (X - 8)$

si no

Hacer $P1 \leftarrow P1 (P1 (X + 9))$

2. {Fin del condicional del paso 1}

8. Escriba un subprograma recursivo que le permita calcular el determinante de una matriz cuadrada de dimensión n .
9. Escriba un subprograma recursivo que busque un valor X en un arreglo unidimensional de enteros, ordenado en forma decreciente.
10. Escriba un subprograma recursivo tal que dado como dato un arreglo unidimensional de enteros positivos de dimensión N , determine si las sumas de las dos mitades (del elemento 1 al $N/2$ y del elemento $N/2 + 1$ al N) son iguales.
11. Escriba un subprograma recursivo que quite todos los espacios en blanco de una cadena de caracteres.
12. Dados como datos dos números enteros positivos A y B —el segundo puede ser también nulo—, escriba un subprograma recursivo que calcule A^B .
13. Escriba un subprograma que resuelva la función de Ackermann en forma iterativa. Compare su solución con la analizada en este libro.
14. Escriba un subprograma recursivo que, dado como dato un número entero positivo, regrese 1 si todos los dígitos de dicho número son pares y 0 en otro caso.
15. Retome el problema anterior y resuélvalo de manera iterativa. Compare sus soluciones, teniendo en cuenta la eficiencia en el manejo de memoria y la legibilidad del código generado.
16. Escriba un subprograma recursivo que, dado como dato un número entero positivo, regrese 1 si el número es divisible por 11 y 0 en otro caso. El criterio que deberá usarse para determinar si es divisible es que la diferencia entre la suma de los dígitos que ocupan posiciones pares y la suma de los dígitos que ocupan posiciones impares sea un múltiplo de 11. Por ejemplo, si el número es 6 479, se tiene que $13(6 + 7) - 13(4 + 9) = 0$ y 0 es múltiplo de 11.
17. Retome el problema anterior y resuélvalo de manera iterativa. Compare sus soluciones, teniendo en cuenta la eficiencia en el manejo de memoria y la legibilidad del código generado.
18. Escriba un subprograma recursivo que, dado como dato un número entero positivo, regrese como resultado la suma de sus divisores.
19. Retome el algoritmo 3.5 —*Conv_postfija*— o el 3.6 —*Conv_prefija*—, del capítulo 3, y desarrolle una versión recursiva del mismo. Compare su solución con la analizada en este libro. Identifique ventajas y desventajas de cada una de ellas.
20. Escriba un subprograma recursivo que invierta el orden de los elementos de una pila. Puede utilizar cualquier estructura de datos como auxiliar, si lo requiere.

21. Escriba un método recursivo, para la clase *Cola*, que imprima todos los elementos de una cola circular.
22. Retome el problema anterior y resuélvalo de manera iterativa. Compare sus soluciones, teniendo en cuenta la eficiencia en el manejo de memoria y la legibilidad del código generado.
23. Escriba un método recursivo, para la clase *Cola*, que invierta el orden de los elementos de una cola. Puede utilizar cualquier estructura de datos como auxiliar, si lo requiere.